

Semantic Web Query Mediation via RDF-entailment under Dynamic Rule Sets

No Author Given

No Institute Given

Abstract. The use of logic-based meta-programming is promising as a declarative framework for automated Semantic Web query mediation in general and over clinical quality reporting and healthcare outcomes patient registries, as a domain-specific example. After capturing a large, patient-care dataset in RDF, measurements on clinical quality and healthcare outcomes can be modeled as RDF-entailment using Datalog-style rule sets. This presents the need to shift from the typical framework of SPARQL querying of an RDF store with materialized components entailed by rules to one in which *both* the query and rules (hence “dynamic rules”) act as input to a stable RDF store *without materialization*. We contribute an application of efficient, goal-directed meta-programming as the basis for implementing distributed SPARQL RIF entailment as a Semantic Web service. We also provide initial experimental results that illustrate the computational advantage of this approach in answering queries that were otherwise infeasible or extremely resource-intensive to attain.

1 Introduction

A *virtual* (or *mediated* [30]) approach to view-based integration is one where queries are decomposed and dispatched against a data source. Such an approach may be better suited than that of materialization for situations where the information sources are changing frequently [17, 31]. Two desired characteristics of a *query mediator* is autonomy and distribution via a communications network [30]. The architectural principles underlying the technologies and standards of the Semantic Web are well attuned to distribution and autonomy and the recent work [24, 27] regarding Semantic Web query federation motivates the need to use traditional deductive database methods to implement Semantic Web *mediators* and *interlocutors*¹.

The recently concluded Rule Interchange Format (RIF) activity within the World Wide Web Consortium (W3C) aimed to develop a standard for exchanging rules among disparate systems, especially on the Semantic Web. Recent work in the World Wide Web Consortium (W3C) has sought to standardize how SPARQL basic graph pattern matching can be extended [16] to incorporate

¹ an interlocutor or middleman is the performer in the middle of a minstrel line who engages the others in talk.

logical entailment in order to provide additional answers to a query beyond those that follow from the structure of the graph alone.

Our method is meant as part of a larger architectural style for SPARQL mediation and entailment that can leverage standardized, Semantic Web rule documentation and ontologies for automated use in a domain of interest. The original motivation was the value - in modern healthcare information systems - of the ability to be able to, on the one hand, query a large amount of longitudinal patient data with a changing collection of adhoc queries that identify a cohort of interest and, on the other hand, generate standard sets of derived variables for the purpose of quality reporting from the same patient data.

In a previous version of the approach discussed here, intermediate quality measures derived from a finite RDF dataset of longitudinal patient records were materialized for use in facilitating quality reporting. The ruleset captured the derivations from the terms in the patient record ontology to various quality measures for the purpose of reporting to the Society of Thoracic Surgeons' (STS) Adult Cardiac Surgery National Database.

Many of these measures were not routinely collected in the registry and had to be derived via a set of rules that represent the semantics of the relationships between them and the base content in the patient registry. An example of two of the rules used to derive common temporal abstractions is captured below in figure 1, rendered using conceptual graph notation [11] for its lucidity. These derived relations were used in the definitions of many of the STS quality measures to capture, for example, the notion of a post-operative (*post-op*) procedure or complication such as a post-op, in-hospital death, i.e., a death that followed a qualifying procedure but occurred within the same hospital episode as the procedure.

The first rule derives a relation between a medical event and the hospital visit it is a part of. The second rule at the bottom derives a relation between two medical events that share a common hospital visit where the beginning of one follows the beginning of the other. This second relation (*startsAfterStartOf*) is transitive.

Our contributions in this paper are: the basis for a rule-based SPARQL entailment regime implementation, efficient, *upside-down* SPARQL meta interpretation, and a query mediation framework.

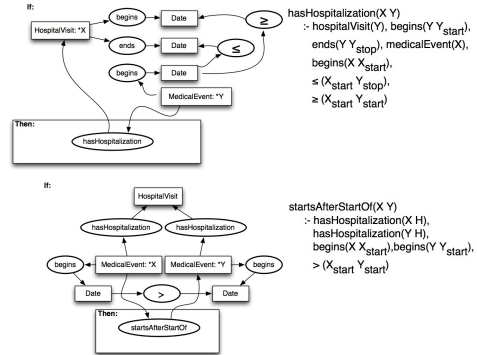


Fig. 1. Rules for deriving hospital episode relations

The paper is organized as follows: section 2 summarizes related work and section 3 introduces a syntax and semantics for basic graph pattern matching. It also introduces the syntax of Datalog, introduces SPARQL, and provides a mechanism for translating between triple patterns and Datalog atomic formula. Section 4 defines our method for rule-based SPARQL entailment. Section 5 discusses an implementation of this method and an experimental evaluation that compares it with a brute force approach. Finally, Section 6 closes with a summary of the experimental results and a comparison against related work.

2 Related Work

Angles and Gutierrez demonstrated [1] that SPARQL has the same expressive power as Relational Algebra under bag semantics. They also showed that *compositional* SPARQL is contained in non-recursive safe Datalog with negation. In [29] and [19] it was demonstrated that SPARQL graph matching can be extended by means of the logical entailment relationships of both Datalog and description logics. In particular, [19] proposes extensions to SPARQL for combining SPARQL querying with dynamic, rule-based inference. They provide additional language constructs for referring to ontologies and rulesets within a SPARQL query.

Chen defined [9] a formal semantics for query answers via query re-writing that preserve the RDFS semantics. As part of a larger solution for browsing RDF data, Lassila proposed [23] an RDF(S) reasoner implemented by means of query re-writing. Makris et al. [25] introduced a mediator-based architecture for integrating information from federated OWL knowledge bases. Jing et al. proposed [20] proposed a method for resolving rewritten SPARQL queries in an original ontology.

In [26], the problem of query rewriting for description logics with certain expressive power restrictions was considered. Their method used a resolution-based technique to produce a rewritten query. Correndo et al. presented [10] an algorithm that exploits *ontology alignments* and *entity co-reference resolution* for rewriting queries over RDF.

3 Background

Before defining a framework for rule-based SPARQL entailment, we need to define the syntax and a semantics of SPARQL and Datalog as well as a mechanism for converting back and forth between the two.

3.1 RDF and Triple Pattern Evaluation

Let I, B, L be pairwise disjoint sets representing IRIs, Blank nodes, and RDF literals, respectively. We denote by T the union $I \cup B \cup L$ (RDF terms). A tuple $(s, p, o) \in (I \cup B) \times I \times (I \cup B \cup L)$ is called an *RDF triple* or *statement*. An RDF triple is *ground* if it does not contain any blank nodes. An *RDF graph* is a set of RDF triples. The notion of ground RDF triple is extended naturally to RDF graphs. For the purpose of the method we present here, we only consider ground RDF graphs. However, this restriction is minor as the use of skolemization as a preprocessing step for RDF graphs to ensure interoperability with rule languages and SPARQL entailment is well documented in the literature [7, 12, 15].

Let V be a set (of variables) disjoint from T . An RDF triple that also makes use of variables V is called a *triple pattern*. A *basic graph pattern* (or BGP for short) is a finite set of triple patterns.

A *solution mapping* (or simply mapping or binding) μ is a partial function $\mu : V \rightarrow T$. The domain of μ , $\text{dom}(\mu)$, is the subset of V where μ is defined. The empty mapping is the everywhere-undefined function with its domain empty. Two mappings μ_1, μ_2 are *compatible*, if $\mu_1(x) = \mu_2(x)$ for all $x \in \text{dom}(\mu_1) \cap \text{dom}(\mu_2)$. If μ_1 and μ_2 are compatible, then $\mu_1 \cup \mu_2$ is again a partial function called their union. When $\mu_j(x) = \mu_i(x)$ for every $x \in \text{dom}(\mu_i)$, we say $\mu_i \leq \mu_j$.

Two triple patterns t_1, t_2 are called *unifiable* if there exists μ such that $t_1[\mu] = t_2[\mu]$, where $t[\mu]$ is instantiation of variables in t according to the mapping specified by μ . μ is called the *unifier* of t_1 and t_2 . We use a set $\uparrow(t_1, t_2)$ to denote all the unifiers of t_1 and t_2 . If t_1, t_2 are unifiable, we write $t_1 \uparrow t_2$. This is similar to the approach of Hung et al. in [18].

Let t be a triple pattern. The interpretation of t with respect to an RDF graph G is defined as $\llbracket t \rrbracket_G = \{(s, p, o) \in G \mid (s, p, o) \uparrow t\}$. Thus, $\llbracket t \rrbracket_G$ represents the set of all edges (triples) of G that are unifiable with t .

Next we extend the definitions to basic graph pattern, i.e., a finite set of triple patterns. Two basic graph patterns T_1, T_2 are called *unifiable* if there exists a *unifier* μ such that $T_1[\mu] = T_2[\mu]$, where $T[\mu] = \{t[\mu] \mid t \in T\}$. If T_1, T_2 are unifiable, we write $T_1 \uparrow T_2$. The set $\uparrow(T_1, T_2)$ include all the unifiers of T_1 and T_2 . The interpretation of a basic graph pattern T with respect to an RDF graph G is defined as $\llbracket T \rrbracket_G = \bigcup \{T_i \subseteq G \mid T_i \uparrow T\}$.

An RDF rule is a rule of the form

$$t \leftarrow T$$

where t is a triple pattern and T is a set of triple patterns. The set that includes all the possible ground triples that can be derived from a set of RDF rules and a graph is called a *closure*, the formal definition of which is given in Definition 1.

Definition 1. Let G be a RDF graph, T the union of $I \cup B \cup L$ (RDF terms), R a set of RDF rules, t a triple pattern, T a set of basic triple patterns, and V a set of variables. The closure of G with respect to R is a graph G^* with the following properties:

- If $(s, p, o) \in G$, then $(s, p, o) \in G^*$,

- For all rule $t \leftarrow T \in R$, if $\exists G' \subseteq G^*$ s.t. $G' \uparrow T$, then $\forall \mu \in \uparrow(G', T). t[\mu] \in G^*$,
- G^* is the smallest graph with the above two properties.

Suppose there is a set R of RDF rules, the operator Γ^R that gets the immediate consequences by applying the rules on a RDF graph G is defined as follows:

$$\Gamma^R(G) = \{t[\mu] \mid \exists t \leftarrow T \in R, G' \subseteq G. s.t. G' \uparrow T \& \mu \in \uparrow(G', T)\}.$$

Based on Γ^R , we can define a sequence of sets $\Gamma_i^R(S) (0 \leq i \leq \infty)$ inductively:

$$\begin{aligned} \Gamma_0^R(S) &= S \\ \Gamma_{i+1}^R(S) &= \Gamma^R(\Gamma_i^R(S)) \cup \Gamma_i^R(S) \end{aligned}$$

$\Gamma_i^R(S) \leq \Gamma_{i+1}^R(S)$ for all $i \geq 0$. Continuing the calculation of $\Gamma_i^R(S)$, we can get the fixpoint of Γ^R on S , denoted as $\Gamma_\omega^R(S)$.

Note, in [19], the closure of G with respect to R is directly defined as $\Gamma_\omega^R(G)$, while in this paper, it is defined in a piecemeal fashion. We can show that the two of them are equal.

Proposition 1. $\Gamma_\omega^R(G)$ is the closure of G with respect to R .

Proof. Please see Appendix.

The method of getting the closure of G with respect to R by calculating $\Gamma_\omega^R(G)$ through a sequence of sets is called the *naive method* in [3].

Using the concept of closure, we can restate the definition of $\mathcal{R}$ -entailment [19].

Definition 2. Let G^* be the closure of G with respect to a set of rules R . Then for any graph $G' \subseteq G^*$, G' is R -entailed by G , denoted as $G \models_R G'$.

3.2 Rule-based SPARQL Entailment

The SPARQL specification defines the syntax and semantics of the recommended query language for RDF. SPARQL is a compositional language that builds on top of basic graph pattern evaluation by providing FILTER expressions that sift out unwanted solution mappings and algebraic operators over sets of solution mappings. As described in [29, 22], entailment regimes that extend the evaluation of basic graph patterns can be used independently from the algebraic manipulation of the solutions provided.

An example of a SPARQL query that makes use of the derived relations presented earlier is in figure 2:

Intuitively, this query returns the closest pre-operative (*pre-op*) blood test that occurs within the same hospital visit with respect to an operation of interest. Although the method described here does not implement FILTER evaluation, it

```
SELECT * {
  data:QualifyingOperation1
  :hasHospitalization ?EPISODE;
  :startsAfterStartOf ?PREOPTEST.
  ?PREOPTEST a ehr:BloodTest;
  :hasHospitalization ?EPISODE.
  OPTIONAL {
    data:QualifyingOperation1
    :startsAfterStartOf ?OTHERTEST.
    ?OTHERTEST a ehr:BloodTest;
    :startsAfterStartOf ?PREOPTEST;
    :hasHospitalization ?EPISODE.
    FILTER(?OTHERTEST != ?PREOPTEST)
  }
  FILTER(!BOUND(?OTHERTEST))
}
```

Fig. 2. Example SPARQL query

can be re-used as part of a larger component that implements the semantics of the SPARQL algebra operators and FILTER expressions over answers provided by the method. In addition, basic graph patterns with variables in the predicate position (i.e., syntactically, higher-order patterns) are outside the scope of this work at this time.

3.3 Transforming Datalog Rules to Triple Patterns

We assume familiarity with the syntax and semantics of Datalog [8]. The rules on the right-hand side of figure 1 are the corresponding Datalog rules for the conceptual graphs.

A definite, function-free logic program P is a set of rules or clauses each with the form

$$p_0(\bar{X}_0) :- p_1(\bar{X}_1), \dots, p_m(\bar{X}_m)$$

where each p_i is a predicate name (or symbol) and $\bar{X}_i$ is a vector of terms associated with p_i , each of which is either a constant or a variable. An *atomic formula* (or atom for short) $p_i(\bar{X}_i)$ is the combination of a name and vector of terms or arguments. Given a rule r , we will use $h(r)$ to denote the atom in the head of r , and $b(r)$ to denote the set of all the atoms in the body of r .

Since we are using Datalog rules but we are querying an RDF graph, there is a mismatch between the syntax of Datalog atoms and triple patterns. So, we need a function to translate back and forth between the two. The function below takes an atomic formula and returns a SPARQL triple pattern:

Definition 3. Given an atomic formula $p(\bar{X})$ of arity $a \in \{1, 2\}$ where $a = |\bar{X}|$, $\mathcal{T}$ is a function that produces triple patterns per [1, 7] as follows:

$$\begin{cases} (s, p, o) & \text{if } a = 2 \text{ and } \bar{X} = (s, o) \\ (member, \text{rdf:type}, p) & \text{if } a = 1 \text{ and } \bar{X} = (member) \end{cases}$$

Typically, as in [7], the non-variable terms of the atoms are drawn from a known RDF graph vocabulary. $\mathcal{T}^{-1}$ is a reverse function that takes a triple pattern and returns a corresponding atom. Also, we define a function $Pred$ that collects the predicate in an RDF triple (s, p, o) as follows: $Pred((s, p, o)) = \begin{cases} p & \text{if } p \text{ is not } \text{rdf:type} \\ o & \text{otherwise} \end{cases}$.

4 SPARQL Interlocution

4.1 A variant of Γ^R

Most of the literature on implementations of rule-based SPARQL entailment leverage an approach that computes derived relationships beforehand, i.e. a *materialization* approach.

However, in a virtual approach to rule-based query answering, it is important that the process of answering queries against a possibly remote RDF graph is idempotent. That is, upon reaching the fixpoint, the RDF graph should be the same as it was before and derived triples should not be inserted into the graph. This allows the component that computes the entailed atoms and the RDF store that the graph is in to remain independent of one another. We give a fixpoint operator I^G specific to an RDF graph G and a set of Datalog rules R , to compute the immediate consequences. The graph is kept outside of the fixpoint set, but the rules are put inside. As a result, the domain only contains Datalog rules and the derived facts:

Definition 4. For any set $\mathcal{D}$ comprised of facts and rules, let $\sigma : V \rightarrow T$ be a solution mapping, $s_0(\bar{x}_0)[\sigma] \in I^G(\mathcal{D})$ if there exists

$$- s_0(\bar{x}_0) :- s_1(\bar{x}_1), \dots, s_n(\bar{x}_n) \in \mathcal{D}, \text{ and}$$

one of the following conditions holds for all $1 \leq i \leq n$:

1. $s_i(\bar{x}_i)[\sigma] \in \mathcal{D}$
2. $s_i \in EDB(G)$, and $s_i(\bar{x}_i)[\sigma] \in \mathcal{T}^{-1}(G)$

Where $EDB(G) = \{Pred(t) \mid t \in G\}$ and $\mathcal{T}^{-1}(G) = \{\mathcal{T}^{-1}(t) \mid t \in G\}$.

If the body atom is a derived predicate(Condition 1), it must have been in the set $\mathcal{D}$ already. Otherwise, it is a database relation(Condition 2) and so we have to query the RDF graph, triggering a SPARQL query to check whether $s_i(\bar{x}_i)[\sigma] \in \mathcal{T}^{-1}(G)$:

1. Translate $s_i(\bar{x}_i)$ to a graph pattern $t = \mathcal{T}(s_i(\bar{x}_i))$.
2. Query t against the graph G , get a set $\llbracket t \rrbracket_G \subseteq G$.
3. Check if $s_i(\bar{x}_i)[\sigma] \in \mathcal{T}^{-1}(\llbracket t \rrbracket_G)$.

Given a RDF graph G and the set of Datalog rules R . Let the initial set be $\mathcal{I}_0 = R$, then $I_\omega^G(\mathcal{I}_0)$ will include all the possible ground atoms that can be derived from the rules in R and facts in $\mathcal{T}^{-1}(G)$, as stated below:

Corollary 1. $\mathcal{T}(I_\omega^G(R)) \cup G$ is the closure of G with respect to R .

Below we give an example. G is an RDF graph and R contains the two rules given in Figure 1:

Example 1. Suppose that after translating the triples in G to atoms, the resulting $\mathcal{T}^{-1}(G)$ is

```
{
  hospitalVisit(b),
  medicalEvent(a), medicalEvent(c), medicalEvent(d),
  begins(a, 2001-4-15), begins(b, 2001-4-1), begins(c, 2001-5-1),
```

```

    begins(d,2001-5-20),
    ends(a,2001-4-16),ends(b,2001-5-31),ends(c,2001-5-2),
    ends(d,2001-5-21)
}

```

In order to answer a ground query $startAfterStartOf(d, a)$, the naive method will calculate the entire closure. As a result, the atoms in the following two sets will be generated (separated by the derived predicates they are instances of):

```

{ hasHospitalization(a,b),hasHospitalization(c,b),
  hasHospitalization(d,b) }
{ startAfterStartOf(d,c), startAfterStartOf(d,a),
  startAfterStartOf(c,a) }

```

Nevertheless, all the triples in G will be queried.

If the database is very large, it becomes expensive to implement the naive method because it exhaustively computes all the possible derivations. In the following section, we provide a meta-computation and mediation method for efficiently computing answers specific to a query, based on a set of recursive Datalog rules, annotated to indicate information regarding the propagation of variable bindings, as a fixpoint procedure.

4.2 Meta Interpreter Rules

Bry developed [5, 6] a sound and complete fixpoint procedure for answering queries over recursive databases based on meta-interpretation called the Backward Fixpoint Procedure (BFP). A *meta-interpreter* for a language is an interpreter for the language written in the language itself [28].

Following Bry's meta interpreter, we replace all the rules $R(G)$ with meta rules $MR(G)$, and denote the fixpoint operator using the meta interpreter rules as Γ^{G_m} .

Definition 5. For each interpreted rule (k) :

$$p(\bar{X}_0) :- q_1(\bar{X}_1), \dots, q_m(\bar{X}_m)$$

produce the following set of rules:

$$p(\bar{X}_0) :- query_b - p(\bar{X}_0), evaluate(k, m, \bar{X}) \quad (1)$$

$$evaluate(k, 0, \bar{X}) :- query_b - p(\bar{X}_0) \quad (2)$$

For $j = 0, 1, \dots, m - 1$:

$$evaluate(k, j + 1, \bar{X}) :- evaluate(k, j, \bar{X}), q_{j+1}(\bar{X}_{j+1}) \quad (3)$$

$$query_b - q_{j+1}(\bar{X}_{j+1}) :- evaluate(k, j, \bar{X}) \quad (4)$$

Where $\bar{X}$ is the set of all unbound variables in the head and body of a rule.

Given a query $p(\bar{X}_0)$, we start from $query_b - p(\bar{X}_0)$, and the computation sequence for each $q_i(X_i)(q_i \notin EDB(G))$ will be:

$$evaluate(k, i-1, \bar{X}_{i-1}) \rightarrow query_b - q_i(\bar{X}_i) \rightarrow \dots \rightarrow q_i(\bar{X}_i) \rightarrow evaluate(k, i, \bar{X}_i),$$

and for every $q_i \in EDB(G)$, the computation sequence is:

$$evaluate(k, i-1, \bar{X}_{i-1}) \rightarrow q_i(\bar{X}_i) \text{ (got by SPARQL Query) } \rightarrow evaluate(k, i, \bar{X}_i),$$

When $q_{j+1} \in EDB$, meta rule (3) is used:

$$evaluate(k, j+1, \bar{X}) :- evaluate(k, j, \bar{X}), q_{j+1}(\bar{X}_{j+1}),$$

Ensuring Execution of Only Relevant SPARQL Queries It is not until there is a binding σ s.t. $evaluate(k, j, \bar{X})[\sigma]$ already exists that we check whether $q_{j+1}(\bar{X}_{j+1})[\sigma] \in \mathcal{T}^{-1}(G)$. This will guarantee that we only evaluate the necessary SPARQL queries. Also, because the query result will generate a binding σ_{j+1} for $q_{j+1}(\bar{X}_{j+1})$, which may be different from σ for $evaluate(k, j, \bar{X})$, the binding passed to $evaluate(k, j+1, \bar{X})$ will be $\sigma \cup \sigma_{j+1}$. Given the manner in which such subordinate SPARQL queries are dispatched and their answers are combined and processed, an implementation of this meta interpreter can be considered a mediator [30, 31].

The Sideways Information Passing (SIP) Strategy Since the computation sequence passes variable bindings along with each atom, it actually implements the Sideways Information Passing Strategy (*sip*), which is a set of decisions on how information gained about tuples in some atoms in a rule is to be used in evaluating subsequent atoms in the rule [4]. As described by Bancilhon et al. in the seminal paper on *magic sets* [2], the applications of interest for *sideways information passing* strategies is the situation where there exists a desirable property in the algorithm used that only *relevant* facts are considered, i.e., such a fact might be essential to the establishment of the answer to the user-specified query. The meta interpreter essentially follows this strategy because the rules are triggered by initial queries and the computation procedure actually indicates a hidden sip.

A sip is used to *adorn* the interpreted program. An adorned atomic formula with predicate name p , corresponds to a computation of the atom with some arguments bound to constants, and the other arguments free, where the bound arguments are those that are indicated as such by the adornment. For example, $p^{bf}(\bar{X})$ corresponds to computing the formula with the first argument bound and the last argument free. So when we use meta rules, we always adorn the predicates first according to the queries.

Since the binding mappings are partial functions, there may be non-ground items, i.e, items with free variables. For those like $evaluate(\bar{X})[\sigma]$, it is safe to

insert them to the fixpoint set (in practice, we use tables to store them, where *NULL* represents free variables), because they are used “inside” rule k , however, for $q \notin EDB$, $query_b - q(a, x)$ is used to trigger another rule k' , which may be a problem because $query_b - q(a, x)$ is syntactically different from $query_b - q(a, y)$, although logically they are the same. So when adorn $query_b - q$, we have to remove the free variables. As a result, both of $query_b - q(a, x)$ and $query_b - q(a, y)$ will be $query_b - q^{bf}(a)$ after adornment.

Extending the SIP Strategy for *Open Queries* Our method differs from that of Bry’s in that it extends sip with the ability to handle open queries. In particular, it provides a mechanism to evaluate a rule from the head to the first (and possibly only) atom in the body in order to provide answers to an open query that matches the head. For meta rule 4 and 2, if the adornment of the corresponding $q_{j+1}(\bar{X}_{j+1})$ or $p(\bar{X}_0)$ does not indicate any bound arguments, then the query predicate created is $query_b - OpenQuery(q_{j+1})$ or $query_b - OpenQuery(p)$, respectively.

4.3 Efficient Computation Using Meta Interpreter Rules

Definition 6. SPARQL Interlocation

Given a RDF graph G and the set of Datalog rules $R(G)$, let $MR(G)$ be the meta rules produced from $R(G)$, and let $Q = \{query_b - p(\alpha) | \alpha \text{ is a vector of terms}\}$ be the query set. Take the initial set as:

$$\mathcal{I}_0^M = MR(G) \cup Q,$$

the process of computing $\Gamma_\omega^{G_m}(\mathcal{I}_0^M)$ is called **SPARQL Interlocation**.

The naive method can be considered as a global approach since it computes all the possible results, while the meta-interpreter is goal-directed, it uses a $query_b - p(\alpha)$ to start the evaluation of a rule, this method can be viewed as a local approach.

In contrast to Example 1, we take the same RDF graph G and rules in Figure 1 and answer the query *startsAfterstartOf*(d, a) via SPARQL Interlocation. The set of meta rules is omitted in the example below to save space.

Example 2. Since $Q = \{query_b - startsAfterstartOf(d, a)\}$, the main computation sequence along the second rule in Figure 1 will be:

$query_b - startsAfterstartOf(d, a)$
 $\rightarrow query_b - hasHospitalization^{bf}(d) \cdots \rightarrow hasHospitalization(d, b)$
 $\rightarrow query_b - hasHospitalization^{bf}(a) \cdots \rightarrow hasHospitalization(a, b)$
 $\rightarrow begins(d, 2001-5-20) \rightarrow begins(a, 2001-4-15)$
 $\rightarrow > (2001-5-20, 2001-4-15)$

$\rightarrow \text{startsAfterstartOf}(d, a)$

From $\text{query}_b\text{-hasHospitalization}^{bf}(d)$ to $\text{hasHospitalization}(d, b)$, the main computation sequence along the first rule in Figure 1 is as follows:

$\text{query}_b\text{-hasHospitalization}^{bf}(d)$
 $\rightarrow \text{hospitalVisit}(b)$
 $\rightarrow \text{begins}(b, 2001-4-1) \rightarrow \text{ends}(b, 2001-5-31)$
 $\rightarrow \text{medicalEvent}(d) \rightarrow \text{begins}(d, 2001-5-20)$
 $\rightarrow \leq (2001-5-20, 2001-5-31)$
 $\rightarrow \geq (2001-5-20, 2001-4-1)$
 $\rightarrow \text{hasHospitalization}(d, b)$

The sequence from $\text{query}_b\text{-hasHospitalization}^{bf}(a)$ to $\text{hasHospitalization}(a, b)$ is also similar. In the fixpoint of this meta computation, the set of newly derived ground atoms is the union of the following two sets:

$\{ \text{hasHospitalization}(a, b), \text{hasHospitalization}(d, b) \}$
 $\{ \text{startAfterStartOf}(d, a) \}$

□

Table 1 shows the comparison between Example 1 and Example 2, in terms of the number of derived atoms and queried atoms. As we can see, although our example only involves two Datalog rules and a very small RDF graph, the difference is still significant.

Table 1. Comparison between Naive SPARQL Entailment and SPARQL Interlocution

	hasHospitalization	startAfterStartOf	EDB Atoms Queried
Naive SPARQL Entailment	3	3	10 (all)
SPARQL Interlocution	2	1	6

The following theorem shows that our strategy is goal-directed. Basically, every derived fact generated for a non-EDB predicate in the BFP fixpoint procedure is an answer to a query, and also, every SPARQL query against the RDF dataset for a EDB predicate is upon request.

Theorem 1. *If $q \notin \text{EDB}$ and $q(\bar{X}_q)[\sigma] \in \Gamma_i^{G_m}(\mathcal{I}_0^M)$, or if $q \in \text{EDB}$ and $q(\bar{X}_q)[\sigma]$ is queried against the RDF dataset in the i 'th step, then there exist $j \leq i$ and a mapping $\sigma_q \leq \sigma$ such that $\text{query}_b\text{-}q(\bar{X}_q)[\sigma_q] \in \Gamma_j^{G_m}(\mathcal{I}_0^M)$.*

Intuitively, the theorem above demonstrate that despite the extensions to Bry's meta interpreter (and sip graphs), SPARQL interlocution still only produces relevant facts and queries if given a priori bindings to propagate but is also able to produce answers if none are given.

The theorem below demonstrates the soundness and completeness of SPARQL Interlocution via equivalence with the answers produced by the naive method.

Theorem 2. Given a RDF graph G , the set of Datalog rules $R(G)$, and a mapping τ , suppose $A = p(\bar{x})$ and A' represents its adorned version according to $query_b - A[\tau]$, let $\mathcal{I}_0 = R(G)$, $\mathcal{I}_0^M = MR(G) \cup \{query_b - A'[\tau]\}$, then

$$A[\tau] \in \Gamma_\omega^G(\mathcal{I}_0) \text{ iff } A'[\tau] \in \Gamma_\omega^{G_m}(\mathcal{I}_0^M)$$

Proof. Please see Appendix.

5 Implementation and Experiments

5.1 Heart and Vascular Institute Implementation

A production rule system for RDF and Datalog-like rules with externally-defined builtin predicates based on the RETE-UL² network [13] was implemented in FuXi³, an open source, logical reasoning (Python) library, similar to (and motivated by) both CWM⁴ and Euler⁵. It was the component of the Heart and Vascular Institute (HVI) reporting system that provided the rule-based RDF entailment capabilities.

Below in figure 3 are two rules used by the system that derive quality measures. These rules make use of the derived predicates defined in section 1. The rule at the top derives a renal failure measure defined as follows in the STS manual: “Indicate whether the patient had acute or worsening renal failure resulting in [...] increase of serum creatinine to > 2.0 [...], and 2x (twice) the most recent preoperative creatinine level.”

The second rule derives a relation used by the first that classifies a event as having happened after a qualifying operation and within a hospital visit common to both.

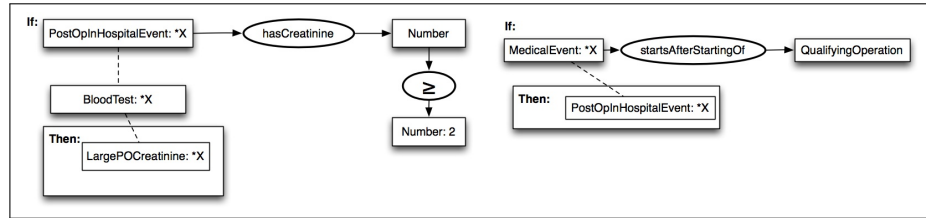


Fig. 3. Rules for deriving quality measures

Overall, the primary bottlenecks in this component of the larger pipeline were: the use of memory to load entire patient record RDF graphs, the large

² The term *Rete* refers to the class of production rule reasoning algorithms that sacrifice memory for computational expedience. It is usually pronounced either *REET* or *REE-tee*, from the Latin word for *network*

³ <http://code.google.com/p/fuxi/>

⁴ <http://www.w3.org/2000/10/swap/doc/cwm.html>

⁵ <http://www.agfa.com/w3c/euler/>

number of (materialized) temporal inferences, the time spent deriving a *renal failure* variable due to the large number of blood test events and their temporal abstractions.

5.2 Experiment Results regarding Goal-directedness

In order to better evaluate the effect of a virtual, goal-directed approach in lieu of these bottlenecks, a quantitative performance comparison was setup between the standard approach and one that leverages SPARQL interlocation. The experiment used an open-source version of the Elliot implementation of SPARQL via SQL query augmentation [14] as the component in the meta-interpreter that evaluates subordinate SPARQL queries to answer top-level triple patterns.

Table 2. Experimental results. TTM - time to materialize each patient record graph (in seconds). TTQ - time to query (in seconds)

	Avg. TTM	Max TTM	Avg. no. of tokens	Max no. of tokens	dataset size	TTQ
Exp. 1	0.766	5	4546	34877	662,838	0.27
Exp. 2	N/A	N/A	N/A	3115	563,279	21

This two-part experiment was built around the derivation of the renal failure variable as defined in the STS database manual and consisted of two processing pipelines for 399 patient records. In the first experiment, entailed patient record graphs were calculated according to the previous manner at the time. A query for all the instances of the *LargePOCreatinine* class was dispatched against the dataset comprised of the closure. In the second experiment, the source patient graphs were loaded directly into the triplestore that received the same query, dispatched against a decision network built from an instance of a meta interpreter for the ruleset and queries on behalf of the triplestore. In addition, the number of partially instantiated *tokens* in the network were recorded. Tokens represent a collection of solution mappings for some subset of the atoms in the body of a rule that has been partially evaluated.

In both cases, the experiment tracked the time spent in the decision networks, the size of the entailed RDF graphs, the most frequently instantiated rules, and the time spent evaluating the queries. The qualifying operations that comprised the report numerator were identified a priori and the results discussed did not include the time it took to do this. Both experiments were run on a Sun Fire X4150 server running Solaris 10 with 4 Quad-Core Intel Xeon processors each clocked at 3.16 GHz and with 4GB of main memory.

On average in the first experiment, there were 250 entailed RDF facts. The largest number of triples entailed by any single patient record graph was 1507.

5.3 Limitations

In the second experiment, dispatching the same open query against a meta-interpreter wrapped around the original RDF dataset took a significant amount

of time to run and the query had to be killed eventually. The primary culprits were the piece-meal evaluation of base atoms, their joins in the meta-interpreter, and of built-in conditions. The same query was run against the meta-interpreter using a ground query. Basically, an **ASK** query was asked to determine if a particular blood test instance (known to fit the criteria) was a member of *Large-POCreatinine* class.

The answer to the query was reached in 21 seconds. However, the amount of tokens involved was significantly less. The meta-interpreter for the unexpanded dataset had 3115 partially instantiated tokens in memory. A majority of these partial instantiations were between the conditions of the meta-interpreter for the rule that derived *hasHospitalization*.

6 Discussions and Conclusions

Amongst the related work that attempts to address the problem of RDF query mediation, there are some similarities to this framework. However, in most cases, the existing methods differ from this framework in the following fundamental ways: most include a proprietary language for capturing the re-writing mechanism that is specified by the user, the rule language is often limited, the query reformulations are primarily structural transformations, and most don't ensure the correctness of the method.

The proposals of Perez-Urbina [26] and that of Ianni et al. [19] are most like the one described here. However, The language targeted by the former is a description logic that is not semantically compatible with Datalog.

In [19], their implementation involves translation from SPARQL, to Datalog, and then to SQL. In ours, the translation is from SPARQL to Datalog and back to SPARQL. Their implementation assumes the database being queried is relational and a mapping from Datalog predicates to tables and views must be provided. In addition, despite the fact that they define a rule-specific entailment relation ($\mathcal{R}$ -entailment), the description of implementation only mentions the use of the SPARQL query as input to the transformation to SQL and doesn't discuss the incorporation of an arbitrary Datalog ruleset to use while evaluating the query. Finally, instantiations of derived, recursive predicates are materialized into a table. Thus, the query operation is not idempotent.

In general, although the meta-interpreter will provide the same answers (and intermediate derivations) as applying a magic set transformed ruleset against an entire database, the way the answers are computed is very different. In the case of the meta-interpreter, sub-queries are generated for idempotent evaluation against a database, which is more suitable for a query mediator. In addition, the meta-interpreter we leverage is a generalization of both the magic set methods as well as top-down methods based on SLD resolution. Finally, leveraging the meta-interpreter allows for the benefits of a backward-chaining procedure without having to implement the complex data structures required to ensure the backward-chaining method will terminate in the face of a set of recursive rules (via memoization).

The experimental results emphasized both the importance of sideways information passing in limiting the answer space of query answering and the impact on performance from a tuple-at-a-time control strategy (such as that of Prolog), which doesn't take advantage of the opportunity to delegate the evaluation of large, connected components of database relations at once.

Possible, future extensions include: support for proof generation via extensions to the meta interpreter, SPARQL FILTER expressions as builtins, evaluating triple patterns where the predicate term is a variable, and for default negation via stable model semantics.

References

1. Angles, R. and Gutierrez, C., 2010. The expressive power of SPARQL. *The Semantic Web-ISWC 2008*, Springer, 114–129.
2. Bancilhon, F. and Maier, D. and Sagiv, Y. and Ullman, J.D., 1985. Magic sets and other strange ways to implement logic programs (extended abstract). *Proceedings of the fifth ACM SIGACT-SIGMOD symposium on Principles of database systems*, 1–15.
3. Bancilhon, F. and Ramakrishnan, R., 1986. An amateur's introduction to recursive query processing strategies. *ACM SIGMOD Record*, 16–52.
4. Beeri, C. and Ramakrishnan, R., 1991. On the power of magic *The journal of logic programming*, 10, 255–299.
5. Bry, F., 1990. Query evaluation in recursive databases: bottom-up and top-down reconciled. *Data and Knowledge Engineering*, 5 (4), 289–312.
6. Bry, F. and Eisinger, N. and Eiter, T. and Furche, T. and Gottlob, G. and Ley, C. and Linse, B. and Pichler, R. and Wei, F., 2007. *Foundations of rule-based query answering. Reasoning Web*, Springer, 1–153.
7. Bry, F. and Furche, T. and Ley, C. and Linse, B. and Marnette, B., 2008. RDFLog: It is like Datalog for RDF. *Proc. Workshop on (Constraint) Logic Programming (WLP)*.
8. Ceri, S., Gottlob, G., Tanca, L., 2002. What you always wanted to know about Datalog (and never dared to ask). *IEEE Transactions on Knowledge and Data Engineering*, 1 (1), 146–166.
9. Chen, H. and Wu, Z. and Wang, H. and Mao, Y., 2006. title=RDF/RDFS-based relational database integration. *IEEE Computer Society*
10. Correndo, G. and Salvadores, M. and Millard, I. and Glaser, H. and Shadbolt, N., 2020. SPARQL query rewriting for implementing data integration over linked data. *ACM, Proceedings of the 2010 EDBT Workshops*
11. Coulondre, S., 1999. CG-SQL: a front-end language for conceptual graph knowledge bases *Knowledge-Based Systems*, Elsevier, 12 (5-6), 293–302.
12. De Bruijn, J., 2010. RIF RDF and OWL Compatibility. *W3C Candidate Recommendation*.
<http://www.w3.org/TR/2010/REC-rif-rdf-owl-20100622/>
13. Doorenbos, R.B., 1995. *Production matching for large learning systems*. Ph.D. thesis.
14. Elliott, B. and Cheng, E. and Thomas-Ogbuji, C. and Ozsoyoglu, Z.M., 2009. A complete translation from SPARQL into efficient SQL. *Proceedings of the 2009 International Database Engineering and Applications Symposium, ACM*, 31–42.

15. Glimm, B. and Krötzsch, M., 2010. SPARQL beyond subgraph matching. The Semantic Web–ISWC 2010, Springer, 241–256.
16. Glimm B., Ogbuji, C., 2011 SPARQL 1.1 Entailment Regimes. W3C Working Draft.
<http://www.w3.org/TR/2011/WD-sparql11-entailment-20110512/>
17. Hull, R. and Zhou, G., 1996. A framework for supporting data integration using the materialized and virtual approaches. ACM Sigmod Record, 25 (2), 481–492.
18. Hung, E. and Deng, Y. and Subrahmanian, V.S., 2005. RDF aggregate queries and views. IEEE, Data Engineering, 2005. ICDE 2005. Proceedings. 21st International Conference, 717–728.
19. Ianni, G. and Krennwallner, T. and Martello, A. and Polleres, A., 2009. Dynamic querying of mass-storage RDF data with rule-based entailment regimes. The Semantic Web–ISWC 2009, Springer, 310–327.
20. Jing, Y. and Jeong, D. and Baik, D.K., 2009. SPARQL graph pattern rewriting for OWL-DL inference queries. Springer, Knowledge and Information Systems, 20 (2), 243–262.
21. Kifer, M., 2008. Rule interchange format: The framework. Springer, Web Reasoning and Rule Systems 1–11.
22. Kolli, I. and Glimm, B. and Horrocks, I., 2011. SPARQL Query Answering over OWL Ontologies. Springer Verlag, Proceedings of the 8th Extended Semantic Web Conference (ESWC 2011).
23. Lassila, O., 2006. Generating Rewrite Rules by Browsing RDF Data. IEEE, Rules and Rule Markup Languages for the Semantic Web, Second International Conference on, 51–57.
24. Langegger, A. and Wölk, W. and Blöchl, M., 2008. A semantic web middleware for virtual data integration on the web. Springer, The Semantic Web: Research and Applications, 493–507.
25. Makris, K. and Bikakis, N. and Gioldasis, N. and Tsinaraki, C. and Christodoulakis, S., 2009. Towards a Mediator Based on OWL and SPARQL. Springer, Visioning and Engineering the Knowledge Society. A Web Science Perspective, 326–335
26. Pérez-Urbina, H. and Horrocks, I. and Motik, B., 2009. Efficient query answering for OWL 2. Springer, Proceedings of The ISWC 2009, 489–504.
27. Quilitz, B. and Leser, U., 2008. Querying distributed RDF data sources with SPARQL. Springer, The Semantic Web: Research and Applications, 524–538.
28. Serebrenik, A., De Schreye, D., 2005. On termination of meta-programs. Theory and Practice of Logic Programming, 5 (03), 355–390.
29. Sirin, E. and Parsia, B., 2007. SPARQL-DL: SPARQL query for OWL-DL . 3rd OWL Experiences and Directions Workshop (OWLED-2007), 4.
30. Wiederhold, G., 1992. Mediators in the architecture of future information systems. IEEE, Computer, 25 (3), 38–49.
31. Widom, J., 1995. Research problems in data warehousing. ACM, Proceedings of the fourth international conference on Information and knowledge management, 25–30.

7 Appendix

Proof of Proposition 1

Let $G^* = \Gamma_\omega^R(G)$, We prove that G^* satisfies the properties in Definition 1:

- $G = \Gamma_0^R(G) \subseteq G^*$.
- For all rule $t \leftarrow T \in R$, if $\exists G' \subseteq G^*$ s.t. $G' \uparrow T$, and $\exists \mu \in \uparrow(G', T)$, then $t[\mu] \in \Gamma^R(G^*) = G^*$.
- Let Q be another graph which satisfies the first two properties. Suppose $\exists g \in G^*$ s.t. $g \notin Q$, then $g \notin G$ because of the first property, which means, there must exist a number n s.t. $g \in \Gamma_n^R(G) \setminus \Gamma_{n-1}^R(G)$. According to the definition, there must exist a rule $t \leftarrow T \in R$ where $g = t[\mu]$, and a mapping μ such that $\exists G' \subseteq \Gamma_{n-1}^R(G)$, $G' \uparrow T$, and $\mu \in \uparrow(G', T)$. It must be the case that $\Gamma_{n-1}^R(G) \not\subseteq Q$, otherwise we will have $g \in Q$ according to the second property. As a result, there must exist a $g_{n-1} \in \Gamma_{n-1}^R(G)$ but $g_{n-1} \notin Q$. Repeatedly, we will have a $g_0 \in \Gamma_0^R(G) = G$ but $g_0 \notin Q$, since $G \subseteq Q$ because Q satisfies the first property, by contradiction, $\forall g \in G^*$, we must have $g \in Q$.

□

Definition 7. For any set $\mathcal{D}$ comprised of facts and rules, $s_0(\bar{x}_0)[\sigma] \in \Gamma^{G_m}(\mathcal{D})$ if there exists

- $s_0(\bar{x}_0) :- s_1(\bar{x}_1), s_i(\bar{x}_i) \in \mathcal{D} (i \leq 2)$,
- $\sigma_i (1 \leq i \leq 2)$ is a mapping,
- $\sigma = \sigma_1 \cup \sigma_2$, and

one of the following conditions holds:

1. $s_i(\bar{x}_i)[\sigma_i] \in \mathcal{D} (1 \leq i \leq 2)$
2. $s_i \in EDB(G)$, and $s_i(\bar{x}_i)[\sigma_i] \in \mathcal{T}^{-1}(G) (1 \leq i \leq 2)$

From the definition, we can see that the computation sequence always passes the bindings to the atoms following it, which is stated by the following Lemma 1:

Lemma 1. If $p_i[\sigma_i]$ precedes $p_j[\sigma_j]$ in one computation sequence, then $\sigma_i \leq \sigma_j$.

Lemma 2. For a rule $k: p(\bar{X}_0) :- q_1(\bar{X}_1), \dots, q_m(\bar{X}_m)$ in $R(G)$, let $MR(k)$ be the set of meta rules produced from k , $\mathcal{I} = MR(k) \cup \{query_b - p(\bar{X}_0)\}$, when computing $\Gamma_\omega^{G_m}(\mathcal{I})$,

- for any $0 \leq i \leq m$, if $query_b - q_i(\alpha)$ was produced, then it should be produced after i 'th step,
- if all $q_i(\alpha) \notin EDB(G)$ were produced after n 'th step, and $q_i(\alpha) \in \mathcal{T}^{-1}(G)$ for other q_i s, then $p(\alpha)$ will be produced in at most $n + m + 2$ steps.

formally,

- if $\exists i'. query_b - q_i(\bar{X}_i)[\sigma] \in \Gamma_{i'}^{G_m}(\mathcal{I})$, then $i' > i$.

- if $q_i(\bar{X}_i)[\sigma] \in \Gamma_n^{G_m}(\mathcal{I})$ for all $q_i \notin EDB(G)$, and $q_i(\bar{X}_i)[\sigma] \in \mathcal{T}^{-1}(G)$ for all $q_i \in EDB(G)$, then $p(\bar{X}_0)[\sigma] \in \Gamma_{n+m+2}^{G_m}(\mathcal{I})$.

Proof of Theorem 1

- When $q \notin EDB$, if $q(\bar{X}_q)[\sigma] \in \Gamma_i^{G_m}(\mathcal{I}_0^M)$, according to the meta rules, there must be a $query_b - q(\bar{X}_q)[\sigma_q]$ which already appeared before q in the computation sequence, which means, there exist $j \leq i$ such that $query_b - q(\bar{X}_q)[\sigma_q] \in \Gamma_j^{G_m}(\mathcal{I}_0^M)$. (That's why we say meta-interpreter is goal-directed.) Also, according to Lemma 1, $\sigma_q \leq \sigma$.
- When $q \in EDB$, suppose q is the n' th element in the body of a rule k , as we mentioned before, it is not until there is a binding σ s.t. $evaluate(k, n-1, \bar{X})[\sigma]$ already exists that we check whether $q(\bar{X}_q)[\sigma] \in \mathcal{T}^{-1}(G)$. So when there is a query for $q(\bar{X}_q)[\sigma]$ against the RDF dataset in the i 'th step, $evaluate(k, n-1, \bar{X})[\sigma]$ must have been generated before, which means $evaluate(k, n-1, \bar{X})[\sigma] \in \Gamma_j^{G_m}(\mathcal{I}_0^M)$ with $j < i$, then according to Meta Rule 4, $query_b - q(\bar{X}_q)[\sigma] \in \Gamma_{j+1}^{G_m}(\mathcal{I}_0^M)$.

□

Proof of Theorem 2

In the following, for each atom $F \notin EDB$, we use F' to represent its adorned version according to the corresponding query.

Necessary condition:

We prove for all $n \geq 0$, all atoms F , mappings τ s.t. $F[\tau]$ is a ground fact, $F[\tau] \in \Gamma_n^G(\mathcal{I}_0)$ implies $\exists m \geq 0$ s.t. $F'[\tau] \in \Gamma_{m+n}^{G_m}(MR(G) \cup \{query_b - F'[\tau]\})$

1) $n = 0$. trivial.

2) $n > 0$. Suppose $F[\tau] \in \Gamma_n^G(\mathcal{I}_0)/\Gamma_{n-1}^G(\mathcal{I}_0)$, there must exist a rule (k') :

$$s_0(\bar{x}_0) :- s_1(\bar{x}_1), \dots, s_k(\bar{x}_k) \in R(G)$$

s.t. $s_0(\bar{x}_0)[\sigma] = F[\tau]$, and for all $1 \leq i \leq k$, one of the following conditions holds:

1. $s_i \notin EDB(G)$, and $s_i(\bar{x}_i)[\sigma] \in \Gamma_{n-1}^G(\mathcal{I}_0)$
2. $s_i \in EDB(G)$, and $s_i(\bar{x}_i)[\sigma] \in \mathcal{T}^{-1}(G)$

If it is condition 1, since $s_0(\bar{x}_0)[\sigma]$ is ground, $s_i(\bar{x}_i)[\sigma]$ must be ground too. Induction hypothesis gives that $\exists m_i \geq 0. s'_i(\bar{x}'_i)[\sigma] \in \Gamma_{m_i+n-1}^{G_m}(MR(G) \cup \{query_b - s'_i(\bar{x}'_i)[\sigma]\})$. From Lemma 2, we know that $query_b - s'_i(\bar{x}'_i)[\sigma] \in \Gamma_{j_i}^{G_m}(MR(G) \cup \{query_b - s'_0(\bar{x}'_0)[\sigma]\})$ with $j_i > i$, as a result, $s'_i(\bar{x}'_i)[\sigma] \in \Gamma_{m_i+n-1+j_i}^{G_m}(MR(G) \cup \{query_b - s'_0(\bar{x}'_0)[\sigma]\})$, let m', j be the maximum of the m_i 's and j_i 's respectively, we have $s'_i(\bar{x}'_i)[\sigma] \in \Gamma_{m'+n-1+j}^{G_m}(MR(G) \cup \{query_b - s'_0(\bar{x}'_0)[\sigma]\})$ for all $s'_i \notin EDB(G)$. Since $s_i(\bar{x}_i)[\sigma] \in \mathcal{T}^{-1}(G)$ for other $s_i \in EDB(G)$ in condition 2, Lemma 2 gives that $F'\tau = s'_0(\bar{x}'_0)[\sigma] \in \Gamma_{m'+n-1+j+k+2}^{G_m}(MR(G) \cup \{query_b - s'_0(\bar{x}'_0)[\sigma]\})$.

Sufficient condition:

We show that for all $n \geq 0$, all adorned atoms F' , mappings τ , $F'[\tau] \in \Gamma_n^{G_m}(MR(G) \cup \{query_b - F'[\tau]\})$ implies $F[\tau] \in \Gamma_n^G(\mathcal{I}_0)$.

1) $n = 0$. trivial.

2) $n > 0$.

Let $\mathcal{I}_F = MR(G) \cup \{query_b - F'[\tau]\}$, suppose $F'[\tau] \in \Gamma_n^{G_m}(\mathcal{I}_F) / \Gamma_{n-1}^{G_m}(\mathcal{I}_F)$, there must exist a rule (k') :

$$p'(\bar{X}'_0) :- query_b - p'(\bar{X}'_0), evaluate(k, m, \bar{X}) \in MR(G)$$

s.t. $p'(\bar{X}'_0)[\sigma] = F'[\tau]$ and $evaluate(k, m, \bar{X})[\sigma] \in \Gamma_{n-1}^{G_m}(\mathcal{I}_F)$. As a result, there also exists a rule (k) :

$$p(\bar{X}_0) :- q_1(\bar{X}_1), \dots, q_m(\bar{X}_m) \in R(G).$$

Since $evaluate(k, m, \bar{X})[\sigma] \in \Gamma_{n-1}^{G_m}(\mathcal{I}_F)$, from the iteration procedure, we get that for all $m_i < m$, $evaluate(k, m_i, \bar{X}_{m_i})[\sigma] \in \Gamma_{n-1}^{G_m}(\mathcal{I}_F)$. Since they were only produced by meta rule 3, we have either 1) $q'_{m_i}(\bar{X}'_{m_i})[\sigma] \in \Gamma_{n-2}^{G_m}(\mathcal{I}_F)$ or 2) $q_{m_i}(\bar{X}_{m_i})[\sigma] \in \mathcal{T}^{-1}(G)$.

For case 1), Theorem 1 shows that there exist $n' \leq n - 2$ and a mapping $\sigma_q \leq \sigma$ such that $query_b - q'_{m_i}(\bar{X}'_{m_i})[\sigma_q] \in \Gamma_{n'}^{G_m}(\mathcal{I}_F)$. So, there must exist

$$q'_{m_i}(\bar{X}'_{m_i})[\sigma] \in \Gamma_{n-2-n'}^{G_m}(MR(G) \cup \{query_b - q'_{m_i}(\bar{X}'_{m_i})[\sigma_q]\}),$$

induction hypothesis gives that $q_{m_i}(\bar{X}_{m_i})[\sigma] \in \Gamma_{n-2-n'}^G(\mathcal{I}_0) \subseteq \Gamma_{n-1}^G(\mathcal{I}_0)$, since $q_{m_i}(\bar{X}_{m_i})[\sigma] \in \mathcal{T}^{-1}(G)$ for other q_{m_i} in case 2), according to rule (k) , we have $F[\tau] = p(\bar{X}_0)[\sigma] \in \Gamma_n^G(\mathcal{I}_0)$. $\square$